# *UNIVERSIDAD AUTÓNOMA GABRIEL RENE MORENO*

## *FACULTAD DE INGENIERÍA Y CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES*




# Ampliación de una ciudad interactiva con OpenGL

**Docente:** KAWAIDA VILLEGAS KENJI

**Integrantes:**

PACO SISCO ROBERTO CARLOS ING.INFORMATICA

**Registro:**

221089772

# Contenido

1. Descripción general del proyecto ..... 3
2. Diseño de la ciudad ..... 3
3. Vehículo principal ..... 3
4. Remolque articulado ..... 4
5. Sistema de colisiones ..... 4
6. Iluminación y modo día/noche ..... 4
7. Cámaras ..... 5
8. Minimapa ..... 5
9. Sistema de entregas ..... 5
10. Controles ..... 5
11. Funciones implementadas ..... 5
12. Organización del código ..... 6
13. Limitaciones ..... 6
14. Tecnologías utilizadas ..... 6
15. Instrucciones de ejecución ..... 7
16. Archivos que deben entregarse ..... 7
17. Recursos y fragmentos externos utilizados ..... 7
18. Comando final ..... 8

## 1. Descripción general del proyecto

El proyecto consiste en una ciudad tridimensional interactiva desarrollada en Java 17 utilizando LWJGL 3.3.3, GLFW y OpenGL 3.3. El objetivo principal es aplicar los conceptos de programación gráfica vistos en la materia mediante la construcción de una escena urbana que pueda ser recorrida con un vehículo controlado por el usuario.

La versión final integra una ciudad completa, sistema de conducción, colisiones, iluminación de día y noche, farolas, faros delanteros, semáforos, minimapa, cámara aérea y cámara de seguimiento, destinos de entrega y un remolque articulado conectado al vehículo principal. Los elementos se generan directamente mediante código, sin utilizar modelos 3D ni texturas externas.

La aplicación se organiza principalmente alrededor de AppCiudad, que contiene la infraestructura gráfica de la ciudad, y AppVehiculo, que extiende esa base para incorporar la conducción, las entregas, las colisiones y el remolque.

## 2. Diseño de la ciudad

La ciudad se construye a partir de una matriz denominada MAPA de 11 × 11 celdas. Cada celda mide 10 unidades, por lo que el área lógica total es de aproximadamente 110 × 110 unidades. Esta matriz permite representar de forma sencilla la distribución urbana y también reutilizar la misma información para detectar zonas por las que el vehículo no puede circular.

- Valor 0: representa una calle o zona destinada a la circulación.
- Valor 1: representa una manzana que contiene un edificio.
- Valor 2: representa un parque.

La distribución produce avenidas continuas entre las manzanas. Los edificios se dibujan mediante cajas tridimensionales y presentan alturas y colores diferentes para evitar que toda la ciudad tenga la misma apariencia. Cada edificio se encuentra sobre una acera y posee ventanas en sus fachadas.

Los parques incorporan césped, árboles y bancos. Los árboles se construyen con un tronco y copas geométricas, mientras que los bancos también se forman con primitivas simples. Además, las calles poseen líneas amarillas discontinuas y pasos peatonales para mejorar visualmente la identificación de las intersecciones.

La escena contiene nueve farolas distribuidas junto a las aceras. Estas no son únicamente elementos decorativos: durante la noche sus posiciones se envían al shader para producir iluminación cercana con atenuación según la distancia.

También se incluyen cuatro semáforos. Su funcionamiento es temporizado y alterna los estados rojo, verde y amarillo. El ciclo visual dura aproximadamente nueve segundos: cuatro segundos en rojo, cuatro en verde y un segundo en amarillo.

## 3. Vehículo principal

El vehículo principal está construido mediante cajas giradas. Se compone de una carrocería, una cabina, cuatro ruedas y dos faros delanteros. Su posición se almacena mediante las variables autoX y autoZ, mientras que la variable angulo representa la orientación del vehículo.

Para determinar hacia dónde debe avanzar se calcula un vector de frente utilizando seno y coseno del ángulo. La nueva posición se obtiene sumando a la posición actual la dirección multiplicada por la velocidad y por deltaTime. El uso de deltaTime permite que el movimiento sea estable aunque cambie la cantidad de cuadros por segundo.

La conducción incorpora aceleración progresiva, retroceso, freno y giro. El giro solo se aplica cuando el vehículo posee una velocidad mínima, lo que evita que rote sobre su propio centro estando completamente detenido.

## 4. Remolque articulado

Como mejora del vehículo principal se incorporó un remolque de carga. El remolque está formado por un piso, dos laterales, una pared trasera, dos ruedas y un enganche delantero en forma de V.

A diferencia de una pieza rígida, el remolque utiliza su propio ángulo, almacenado en anguloRemolque. Cuando el vehículo gira se calcula la diferencia entre el ángulo del auto y el ángulo del remolque. Esa diferencia se aplica de manera gradual, produciendo el efecto de una articulación.

El punto de unión se calcula detrás del vehículo. Desde ese punto se obtiene la posición del centro del remolque siguiendo la dirección indicada por anguloRemolque. De esta forma, al girar, el vehículo cambia primero de dirección y el remolque lo sigue progresivamente.

El enganche en V se construye mediante dos brazos que parten del mismo punto de articulación y llegan a los extremos delanteros de la caja del remolque. Para dibujar cada brazo se calcula la distancia entre sus dos extremos, su punto medio y el ángulo necesario para orientar una caja delgada entre ambos puntos.

## 5. Sistema de colisiones

El método puedeCircular() comprueba si una posición es válida para el vehículo. Primero verifica que el auto no salga de los límites de la ciudad. Después recorre la matriz MAPA y comprueba las celdas ocupadas por edificios o parques. Para evitar que el centro del vehículo llegue demasiado cerca de una manzana se utiliza un radio de seguridad.

El remolque posee una comprobación adicional mediante puedeCircularRemolque(). Este método calcula la dirección del auto y del remolque, obtiene el punto de enganche situado detrás del vehículo y finalmente calcula el centro actual del remolque.

La posición calculada del remolque se envía nuevamente a puedeCircular(). De esta manera se reutiliza el sistema de colisiones existente en lugar de repetir todo el código. El vehículo solo actualiza su posición cuando puedeCircular() y puedeCircularRemolque() devuelven verdadero. Si el auto o el remolque chocan, la velocidad y el acelerador se colocan en cero.

La distancia de 2.6 unidades utilizada en el cálculo del centro del remolque representa la separación aproximada entre el punto de enganche y el centro geométrico de la caja del remolque.

## 6. Iluminación y modo día/noche

La escena implementa dos ambientes principales: día y noche. La tecla N alterna entre ambos estados. El fragment shader combina iluminación ambiente y difusa para determinar el color final de los objetos.

Durante la noche se reduce la iluminación general y se vuelven más visibles las fuentes artificiales. Las nueve farolas actúan como luces puntuales y utilizan atenuación, por lo que su intensidad disminuye a medida que aumenta la distancia entre la luz y el fragmento.

Las ventanas de los edificios utilizan emisión visual durante la noche, lo que produce el efecto de ventanas encendidas. Los semáforos y la marca del destino también pueden utilizar emisión para destacar visualmente.

El vehículo posee dos faros delanteros. Sus posiciones se calculan a partir de la posición, orientación y dirección lateral del vehículo. En el shader se utilizan la dirección de los faros, la distancia y el ángulo del cono para limitar la zona iluminada delante del auto. La tecla F permite encenderlos o apagarlos.

## 7. Cámaras

El proyecto dispone de dos formas principales de observar la escena. La primera es una cámara aérea/orbital que permite visualizar una parte amplia de la ciudad. La segunda es una cámara de seguimiento ubicada detrás del vehículo.

La tecla C alterna entre ambas cámaras. En la cámara de seguimiento, el ojo se coloca detrás y por encima del auto, mientras que el objetivo se sitúa varios puntos delante del vehículo. Debido a que ambas posiciones dependen de la orientación del auto, la cámara acompaña sus giros.

## 8. Minimapa

El minimapa se representa simultáneamente en una esquina de la ventana. Para generarlo se realiza otra representación de la escena desde una vista superior, utilizando una región independiente de la ventana.

El minimapa permite observar la distribución general de la ciudad y ayuda a orientarse durante las entregas. La tecla M permite mostrarlo u ocultarlo. Su tamaño se calcula a partir de las dimensiones de la ventana para conservar una presentación compacta.

## 9. Sistema de entregas

La actividad principal consiste en visitar cuatro destinos distribuidos por la ciudad. Cada destino se almacena mediante coordenadas X y Z y el destino actual se representa con una baliza verde.

En cada actualización se calcula la distancia entre el vehículo y el destino mediante la distancia euclidiana. Cuando la distancia es menor al rango establecido, la entrega se considera completada y se activa el siguiente destino.

Al finalizar los cuatro destinos se marca la actividad como terminada, se detiene el vehículo y el título de la ventana informa que las entregas fueron completadas. La tecla R, y adicionalmente K, permite reiniciar el recorrido desde el principio.

## 10. Controles

| Tecla | Acción |
|---|---|
| W / Flecha arriba | Acelerar y avanzar. |
| S / Flecha abajo | Reducir la marcha y retroceder. |
| A / Flecha izquierda | Girar hacia la izquierda. |
| D / Flecha derecha | Girar hacia la derecha. |
| Barra espaciadora | Aplicar el freno. |
| C | Alternar cámara aérea y cámara de seguimiento. |
| F | Encender o apagar los faros delanteros. |
| N | Alternar entre modo día y modo noche. |
| M | Mostrar u ocultar el minimapa. |
| R / K | Reiniciar el vehículo, el remolque y las entregas. |
| ESC | Cerrar la aplicación. |

## 11. Funciones implementadas

- Renderizado con vertex shader y fragment shader GLSL 330.
- Geometría 3D generada a partir de un cubo base con VAO y VBO.
- Ciudad de 11 × 11 celdas con calles, edificios y cuatro parques.
- Edificios con diferentes alturas, colores, cornisas y ventanas.
- Parques con árboles y bancos.
- Marcas viales y pasos peatonales.
- Cuatro semáforos con ciclo visual rojo, verde y amarillo.
- Modo día y noche.

- Nueve farolas con iluminación puntual y atenuación.
- Dos faros delanteros orientados con el vehículo.
- Vehículo con aceleración, retroceso, frenado y giro.
- Remolque articulado con enganche en V.
- Colisión del vehículo y del remolque.
- Cámara aérea y cámara de seguimiento.
- Minimapa activable.
- Cuatro destinos de entrega.
- Reinicio completo del recorrido.
- Información de velocidad, iluminación, destino y entregas en el título de la ventana.

## 12. Organización del código

El proyecto se separa en clases para mantener responsabilidades claras y facilitar las modificaciones.

| Clase | Responsabilidad |
|---|---|
| AppCiudad | Inicialización de GLFW/OpenGL, shaders, cubo base, ciudad, iluminación y minimapa. |
| AppVehiculo | Controles, movimiento, cámara, colisiones, entregas, faros y lógica del remolque. |
| Vehiculo | Dibujo del auto, ruedas, faros, remolque, enganche y brazos de articulación. |
| Ciudad | Elementos urbanos como parques, árboles, bancos, ventanas, farolas y pasos peatonales. |
| Iluminacion | Posiciones y preparación de las farolas. |
| Semaforo | Dibujo y temporización de los semáforos. |
| Minimapa | Visibilidad, tamaño y posición del minimapa. |

## 13. Limitaciones

- La geometría utiliza principalmente cubos y colores; no se utilizan modelos 3D externos ni texturas.
- No se implementan sombras proyectadas ni reflejos avanzados.
- La física del remolque es una aproximación mediante suavizado angular, no un motor físico real.
- La colisión del remolque se aproxima mediante su posición central, por lo que sus esquinas no se comprueban de forma independiente.
- Los semáforos son visuales y no obligan automáticamente al vehículo a detenerse.
- Las posiciones de elementos urbanos están definidas directamente en el código.
- La versión está preparada principalmente para Windows mediante las dependencias nativas configuradas en Maven.
- No se implementaron sonidos ni modelos externos.

## 14. Tecnologías utilizadas

| Tecnología | Uso |
|---|---|
| Java 17 | Lenguaje principal del proyecto. |
| LWJGL 3.3.3 | Enlace de Java con las bibliotecas gráficas nativas. |
| OpenGL 3.3 | Renderizado tridimensional. |
| GLFW | Ventana, contexto gráfico y entrada por teclado. |
| GLSL 330 | Vertex shader y fragment shader. |
| Apache Maven | Gestión de dependencias, compilación y ejecución. |
| VS Code | Entorno utilizado para editar y ejecutar el proyecto. |

## 15. Instrucciones de ejecución

Requisitos: JDK 17 o superior, Apache Maven 3.x, Windows de 64 bits y una tarjeta gráfica compatible con OpenGL 3.3.

1. Abrir PowerShell o la terminal de VS Code.

2. Entrar a la carpeta del proyecto, por ejemplo: D:\GRAFICA\ProyectoGrafica\opengl-java-class

3. Compilar el proyecto:

```
mvn clean compile
```

4. Ejecutar la versión final:

```
mvn exec:java '-Dexec.mainClass=com.graphics.AppVehiculo'
```

Al ejecutar AppVehiculo se inicia la versión que incluye la ciudad, el vehículo, el remolque, las entregas, las cámaras, la iluminación y el minimapa.

## 16. Archivos que deben entregarse

Para cumplir con la entrega se debe incluir el proyecto fuente completo, especialmente la carpeta src, el archivo pom.xml y cualquier recurso necesario. No es necesario incluir dependencias descargadas por Maven ni archivos temporales generados durante la compilación.

- Código fuente Java del proyecto.
- Archivo pom.xml.
- Recursos utilizados por el proyecto, si existieran.
- Este documento breve.
- Comando exacto de ejecución de la versión final.

Se recomienda excluir carpetas de compilación como target y archivos temporales del entorno de desarrollo.

## 17. Recursos y fragmentos externos utilizados

Para el desarrollo del proyecto se utilizaron los siguientes recursos:

| Recurso | Uso dentro del proyecto |
|---|---|
| **Java 17 / OpenJDK 17** | Lenguaje principal utilizado para programar las clases, movimiento, colisiones, controles, vehículos y lógica general. |
| **LWJGL 3.3.3** | Biblioteca que permite utilizar OpenGL y GLFW desde Java. Se utiliza como enlace entre Java y las funciones gráficas nativas. |
| **OpenGL 3.3** | API gráfica utilizada para renderizar la ciudad en 3D, incluyendo edificios, calles, parques, vehículo, remolque, farolas y demás objetos. |
| **GLFW** | Utilizado para crear y administrar la ventana, recibir entradas del teclado y mantener el contexto de OpenGL. |
| **GLSL 330** | Lenguaje utilizado para los shaders. El *vertex shader* procesa las posiciones y transformaciones, mientras que el *fragment shader* calcula colores e iluminación. |
| **Apache Maven** | Utilizado para organizar el proyecto, descargar las dependencias de LWJGL, compilar el código y ejecutar la aplicación. |

| Recurso | Uso dentro del proyecto |
|---|---|
| **pom.xml** | Archivo de configuración de Maven donde se encuentran las dependencias y configuración necesarias para ejecutar el proyecto. |
| **Visual Studio Code** | Editor utilizado para desarrollar, modificar y ejecutar las clases Java del proyecto. |
| **PowerShell / Terminal** | Utilizado para ejecutar los comandos Maven de compilación y ejecución. |
| **Código base proporcionado en clase** | Las primeras prácticas, como `App.java`, `AppMovimientoTeclado.java` y `AppZoom.java`, sirvieron como referencia/base para la creación de la ventana, shaders, controles y ciclo de renderizado. |
| **Primitivas geométricas propias** | Los edificios, vehículo, remolque, árboles, bancos, semáforos y demás elementos se construyeron mediante geometría generada por código, principalmente cubos transformados. |

## 18. Comando final

```
mvn exec:java '-Dexec.mainClass=com.graphics.AppVehiculo'
```